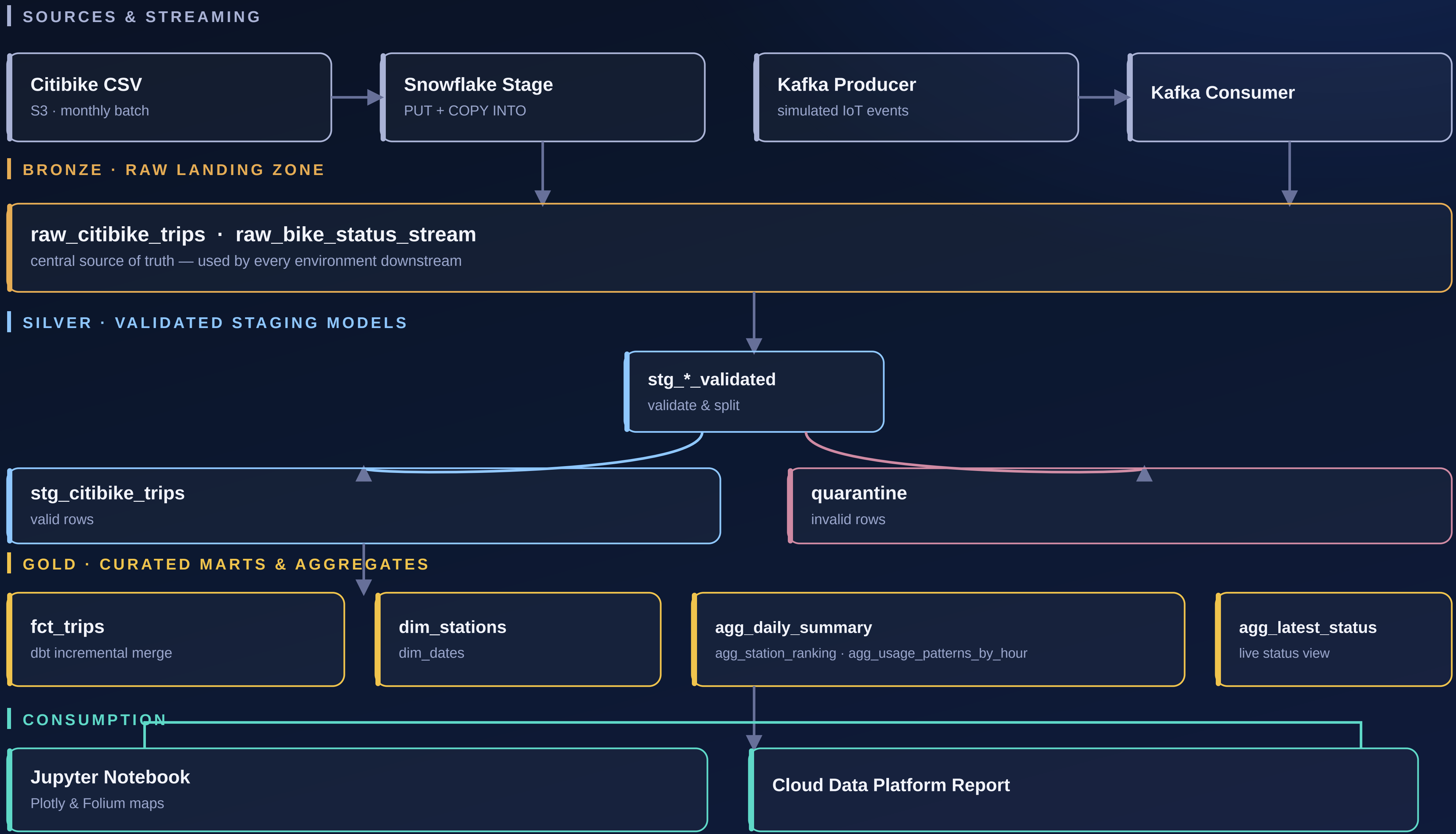
CLOUD DATA PLATFORM · REFERENCE ARCHITECTURE

Jersey City Bikeshare

Streaming & batch ingestion → Snowflake medallion architecture → dbt → analytics



Two repositories, one source of truth

The dbt project lives in exactly one place and is referenced as a git submodule from both the Airflow repo root and its `include/` directory — no manual copies to keep in sync. Four Snowflake databases and three warehouses are fully reproducible from a single `terraform apply`.

TWO GIT REPOSITORIES

jersey-city-bikeshare/

```
▸ .github/workflows/
    · dbt-ci.yml
    · kafka-ci.yml
▸ jersey_city_bikeshare_infra/
    · main.tf · resources.tf
▸ jersey_city_bikeshare_streaming/
    · producer.py · consumer.py
⊗ jersey_city_bikeshare_dbt/    submodule
```

jersey_city_bikeshare_airflow/

```
▸ dags/
⊗ include/.../dbt/    submodule
```

one dbt project — referenced twice, never copied

SNOWFLAKE DATABASES — PROVISIONED END-TO-END VIA TERRAFORM

JERSEY_CITY_BIKESHARE
raw · central source of truth
used by ALL environments

→ **_DEV**
DBT_WH · local dev

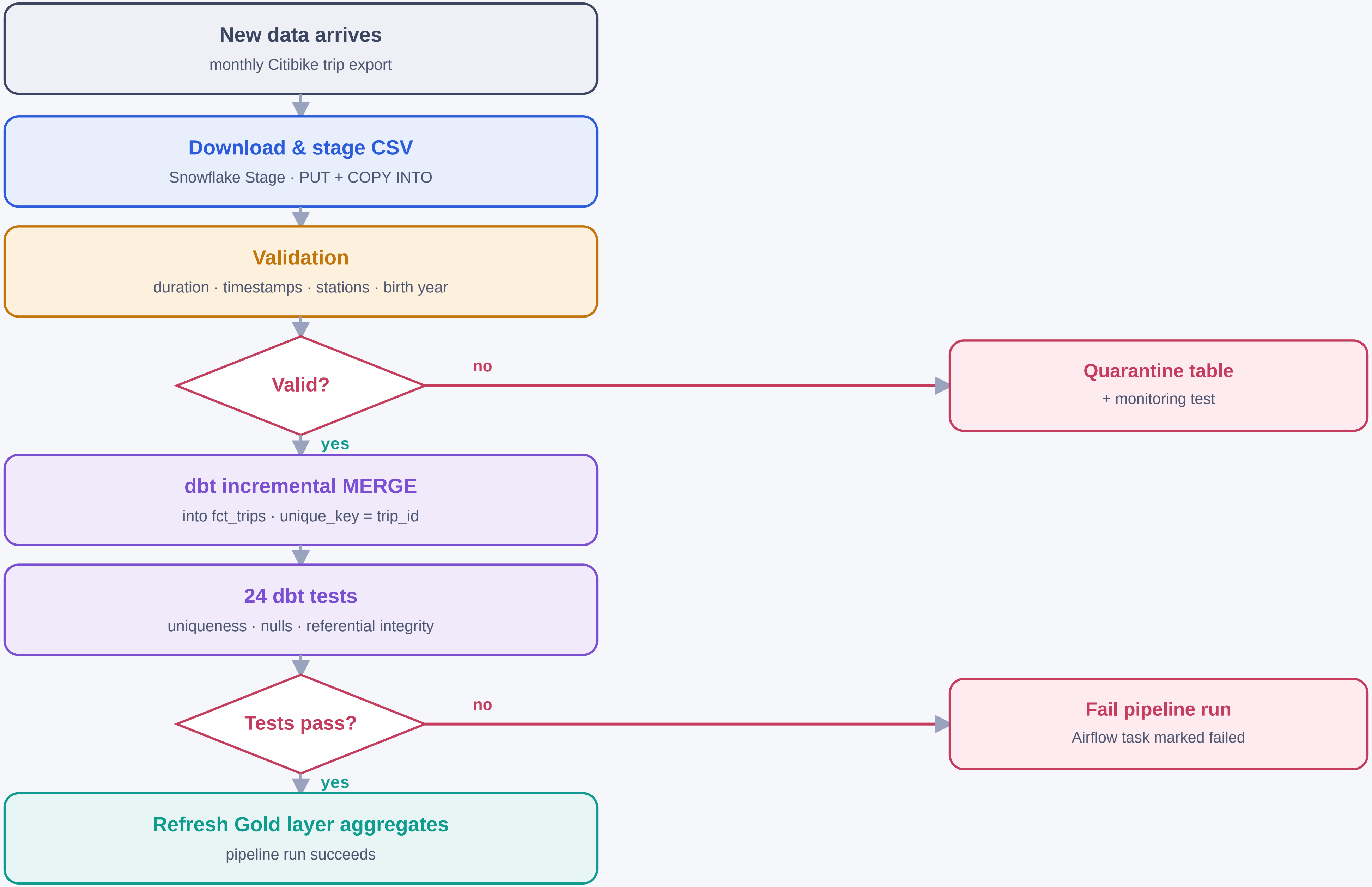
→ **_CI**
CI_WH · GitHub Actions

→ **_PROD**
DBT_WH / BI_WH · analytics

4 databases · 3 warehouses · 1 role · 8 grants — fully reproducible from `terraform apply`

Incremental batch load

Every new monthly CSV is staged, validated, and merged idempotently — rows that fail validation are quarantined rather than dropped, and a failing dbt test stops the run before Gold is refreshed.



Two independent CI/CD pipelines

Path filters mean changing the dbt project never re-runs the Kafka test, and vice versa. Both workflows also support manual `workflow_dispatch`; secrets are injected at runtime and never stored in the repository.

dbt-ci.yml

trigger: push → jersey_city_bikeshare_dbt/**

1. **pip install dbt-snowflake**
2. **dbt deps**
3. **dbt run**
   `--target ci`
4. **dbt test**
   `--target ci`

Runs against isolated CI database
JERSEY_CITY_BIKESHARE_CI

kafka-ci.yml

trigger: push → jersey_city_bikeshare_streaming/**

1. **spin up Kafka service container**
   `wait for broker ready`
2. **producer_ci.py**
   `emits 25 test events`
3. **consumer_ci.py**
   `writes events into Snowflake`
4. **verify event count**
   `fail build if arrived < expected`

Build fails if event count is short
protects the streaming contract

Both workflows also support manual **`workflow_dispatch`** · path filters keep dbt and Kafka CI fully independent · secrets are injected at runtime, never stored in the repo.

Two clearly distinguishable user groups

240,538 trips · August 2018 – March 2019 · processed end-to-end through the medallion pipeline above.

Dataset overview

METRIC	VALUE
Period	August 2018 – March 2019 (8 months)
Total trips	**240,538**
Originally loaded core dataset	Aug 2018 – Jan 2019 (~198,367 trips), later extended
Held back for pipeline testing	Feb + Mar 2019 (42,171 trips) — deliberately withheld, then loaded later to simulate a real "new data arrives" production run
Stations	**87**
Quarantined (invalid) records	**68** — all due to the sentinel value `birth_year = 1888`

~95% Subscribers — commuters

- Two sharp peaks: **7–9 AM** & **5–7 PM** on weekdays
- Short, tightly clustered trips — peak around **250–300 sec** (4–5 min)
- Concentrated Mon–Fri; only **~18%** of volume falls on weekends
- Almost never leave the system — cross-system rate **0.026%**

~5% Customers — leisure / tourist

- Peak **Sunday 10 AM–3 PM**, secondary Saturday-evening hotspot
- Longer, widely spread durations — many trips **800–2,000+ sec**
- Weekend trips make up **~41%** of this group's volume
- **~9× more likely** to end outside Jersey City — cross-system rate 0.23%

Spatial findings

Grove St PATH is by far the busiest station (~7,000 trips start+end) — a transit hub, reinforcing the commuter hypothesis.

A handful of stations sit in Manhattan/Brooklyn since the network is technically interconnected — yet cross-system trips account for only **0.037%** of all trips overall.

Data quality

All **24 automated dbt tests** (uniqueness, nulls, referential integrity, monitoring) pass across the full dataset. **68 records** were correctly routed to quarantine rather than polluting downstream aggregates.

Runtime stayed nearly constant (**~8 seconds**) when volume increased ~9× (24,910 → 229,554 rows) — incremental loading scales as designed.

What I'd change next

Subscribers and Customers use almost complementary time windows — that gap, plus a small data-quality artifact, points directly at four concrete next steps.

Business implications, side by side

	SUBSCRIBER (COMMUTER)	CUSTOMER (LEISURE)
When	Mon–Fri, 7–9 AM & 5–7 PM	Sat/Sun, 10 AM–3 PM
Trip duration	Short, tightly clustered (~4–5 min)	Longer, widely spread
Weekend share	~18% of volume	~41% of volume
Cross-system rate	0.026%	0.23%

1

Differentiate pricing by usage pattern

Subscribers and Customers occupy almost complementary time windows. Skew Subscriber plans toward commute reliability — e.g. a rush-hour flat rate, annual passes focused on weekdays. Skew Customer plans toward weekend/day passes, marketed directly at tourists (e.g. bundled with sightseeing).

2

Rebalance bike capacity by time of day

Bike distribution today is presumably static, but demand is clearly predictable from the data. A rebalancing routine could shift bikes toward residential areas on weekday mornings, toward Grove St PATH and other transit hubs in the evening, and toward leisure hotspots on weekends.

3

Fix the sentinel value at the source

68 records carry `birth_year = 1888`, almost certainly a sign-up form defaulting an optional field instead of allowing it to be left blank. The quarantine layer correctly isolates these rows today — but the real fix belongs further upstream, in the sign-up form itself.

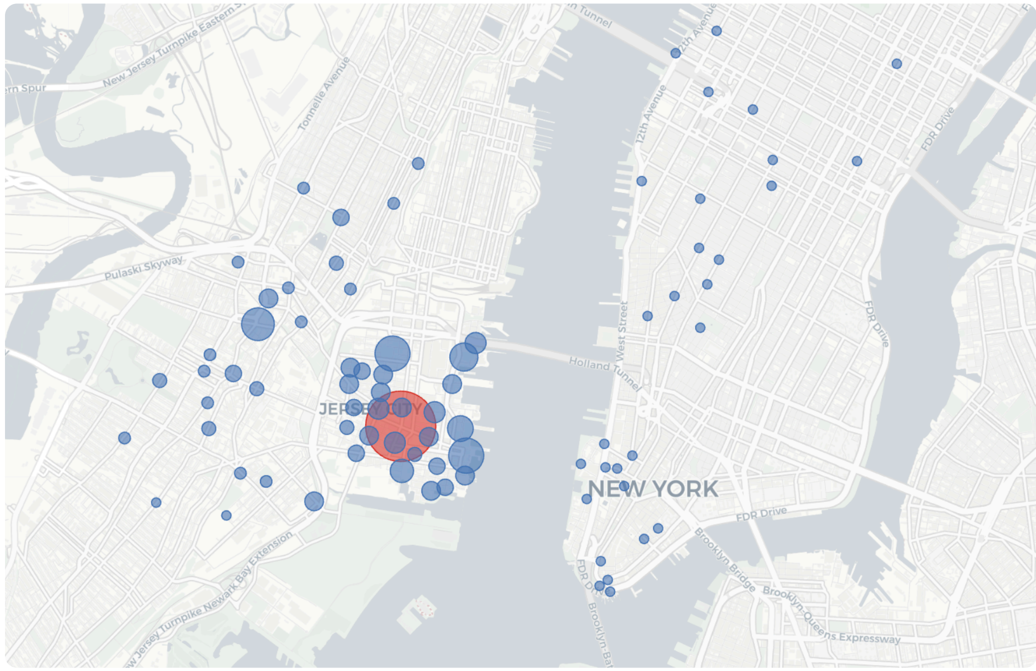
4

Look closer at cross-system trips

At 0.037% of all trips, cross-system usage is too rare to be an investment priority on its own. But Customers do it 9× more often than Subscribers — if that's a real travel pattern rather than noise, an "NYC day-trip" offer could turn an edge case into a marketed feature.

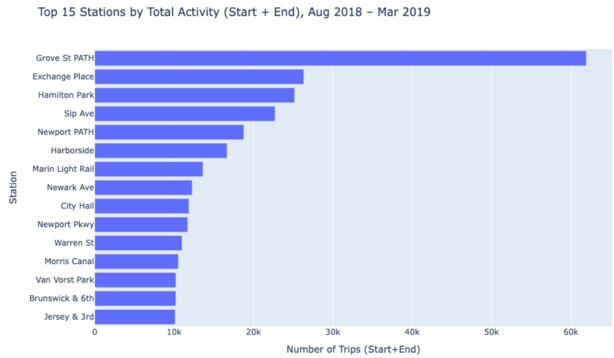
Where the trips happen

Activity is heavily concentrated around the Jersey City core, with a thin secondary cluster across the river in Manhattan.



Station activity — bubble size = trip volume, Jersey City highlighted

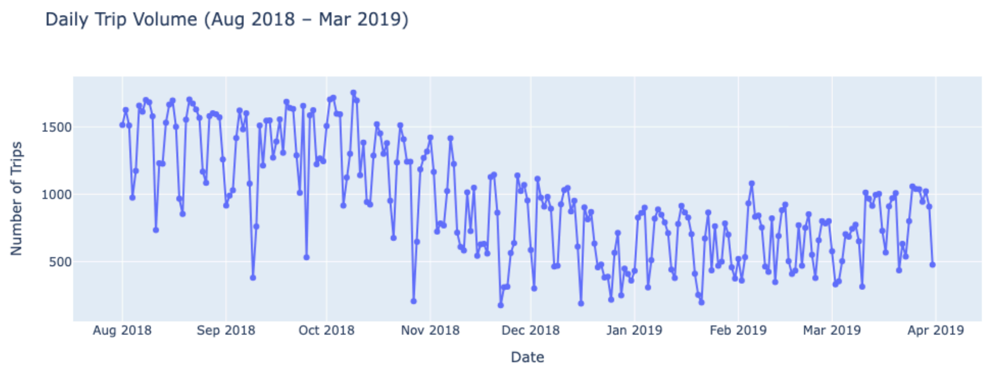
Trip density heatmap across the Jersey City / Manhattan area



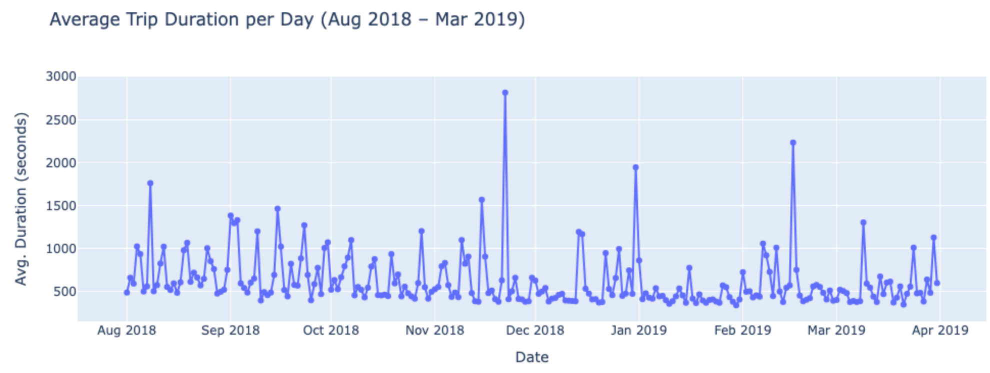
Top 15 stations by total activity (start + end), Aug 2018 – Mar 2019

When and how people ride

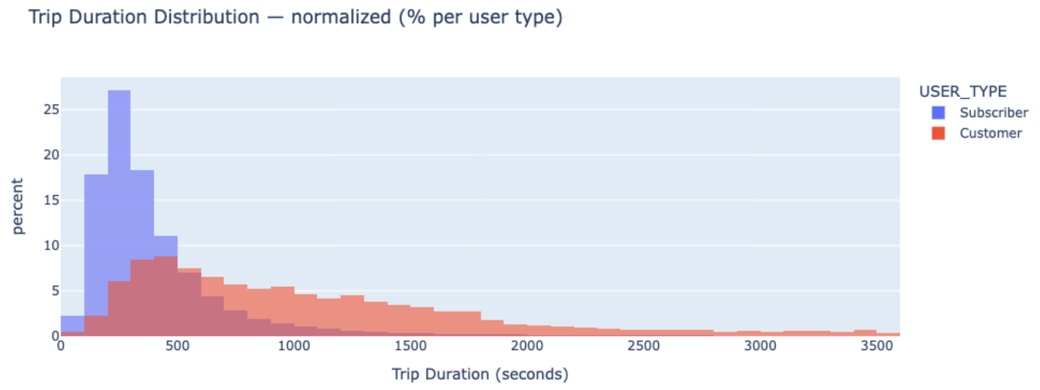
Daily volume, trip duration, and weekday mix all point the same direction: commuters on weekdays, leisure riders on weekends.



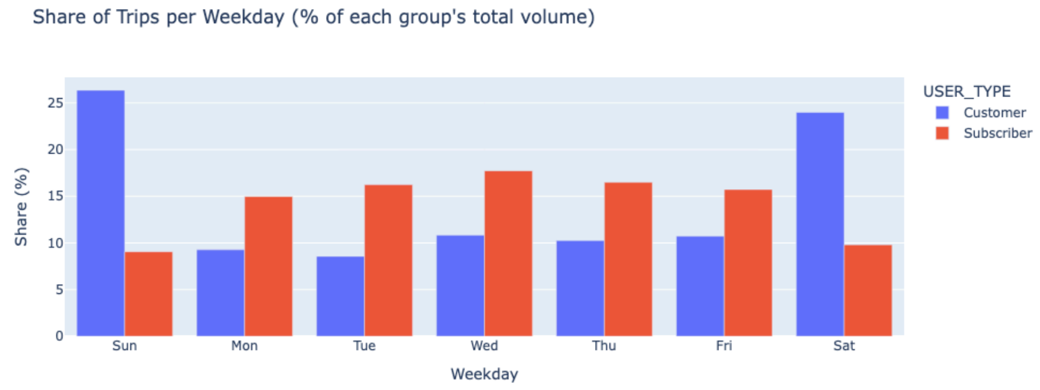
Daily trip volume (Aug 2018 – Mar 2019)



Average trip duration per day



Trip duration distribution by user type (normalized)



Share of trips per weekday, by user type